An Intro to GStreamer

Presented by
Kowshic Shankar R

What is GStreamer?

- GStreamer is a library for constructing graphs of media-handling components.
- The applications it supports range from simple video/audio playback, audio/video streaming to complex audio and video processing.
- Graph-based structure allows arbitrary pipeline construction.
- Dynamically loaded plugins provide elements and media types.
- Element interface handles all known types of sources, filters and sinks.
- Autoplugging uses capabilities system to complete complex paths automatically

What is GStreamer?

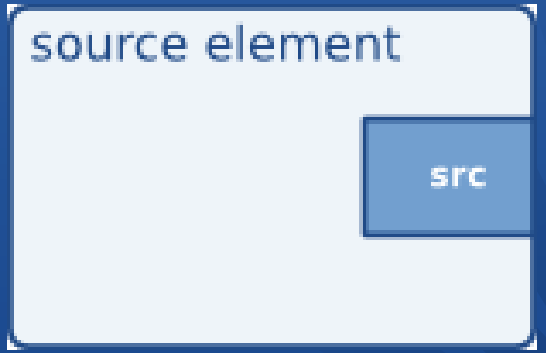
- Pipelines can be visualised by dumping them to a .dot file and creating a PNG image from that .
- Capabilities system allows verification of element compatibility using MIME types and media-specific properties.
- Developers can add new codecs and filters by writing a simple plugin with a clean, generic interface.
- Supported Codecs: FFmpeg, various codec libraries, 3rd party codec packs
- Supported Streaming: http, mms, rtsp

What are Elements?

- An Element is the basic building block for a media pipeline.
- Elements are best visualized as black boxes. On the one end, you might put something in, the element does something with it and something else comes out at the other side.
- For a decoder element, for example, you'd put in encoded data, and the element would output decoded data.

Source elements

- Source elements generate data for use by a pipeline, for example reading from disk or from a sound card.
- Source elements do not accept data, they only generate data.
- You can see this in the figure because it only has a source pad (on the right).
- A source pad can only generate data.



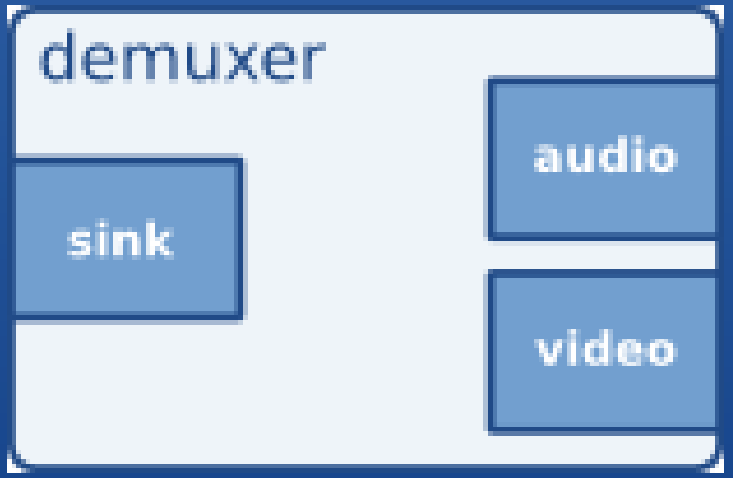
Filter-like elements

- Filter-like elements have both input and outputs pads. They operate on data that they receive on their input (sink) pads, and will provide data on their output (source) pads.
- Examples of such elements are a volume element (filter), a video scaler (convertor), an Ogg demuxer or a Vorbis decoder.



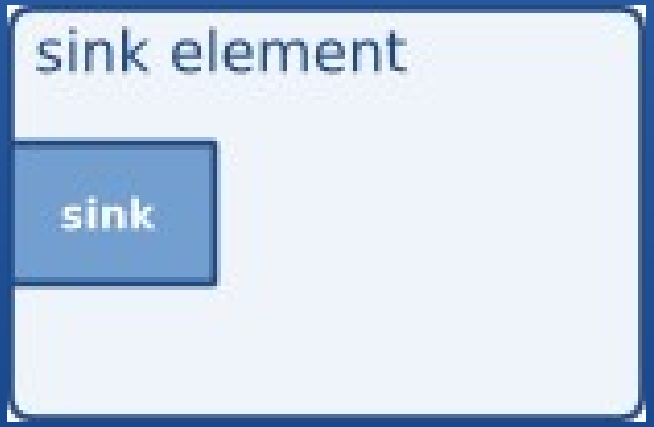
Filter-like elements

- Filter-like elements can have any number of source or sink pads.
- A video demuxer, for example, would have one sink pad and several (1-N) source pads, one for each elementary stream contained in the container format.



Sink elements

- Sink elements are end points in a media pipeline.
- They accept data but do not produce anything.
- Disk writing, soundcard playback, and video output would all be implemented by sink elements.



Media Pipeline

- By linking a source element with zero or more filter-like elements and finally a sink element, you set up a media pipeline.
- Data will flow through the elements.
- This is the basic concept of media handling in GStreamer.



Pads

- Pads are the element's interface to the outside world.
- Data streams from one element's source pad to another element's sink pad.
- The specific type of media that the element can handle will be exposed by the pad's capabilities.
- GStreamer defines two pad directions: source pads and sink pads.
- A pad can have any of three availabilities: always, sometimes and on request.
- Some elements might not have all of their pads when the element is created.

Capabilities of a pad

- Capabilities are attached to pad templates and to pads.
- For pad templates, it will describe the types of media that may stream over a pad created from this template.
- For pads, it can either be a list of possible caps, in which case the pad is not yet negotiated, or it is the type of media that currently streams over this pad, in which case the pad has been negotiated already.
- Properties are used to describe extra information for capabilities. A property consists of a key (a string) and a value.

gst-inspect-1.0

- gst-inspect-1.0 is a tool that prints out information on available GStreamer plugins, information about a particular plugin, or information about a particular element.
- When executed with no PLUGIN or ELEMENT argument, gst-inspect-1.0 will print a list of all plugins and elements together with a sumary.
- When executed with a PLUGIN or ELEMENT argument, gst-inspect-1.0 will print information about that plug-in or element.
- It is used to learn about a Element’s pad templates and capabilities.

gst-launch-1.0

- gst-launch-1.0 is a tool that builds and runs basic GStreamer pipelines.
- Please note that gst-launch-1.0 is primarily a debugging tool.
- You should not build applications on top of it.
- Synopsis:

    gst-launch-1.0 [OPTIONS] PIPELINE-DESCRIPTION

Capturing a Image to a File

```
gst-launch-1.0 \
v4l2src device=/dev/video0 num-buffers=1 ! \
video/x-raw,format=YUY2,width=640,height=480 ! \
filesink location=frame.yuyv
```

- v4l2src can be used to capture video from v4l2 devices, like webcams and tv cards.
- Second element is known as caps filter , it negotiates for the parameters
- Write incoming data to a file in the local file system.

Capturing a Image to a File

```
gst-launch-1.0 \
v4l2src device=/dev/video0 num-buffers=1 ! \
image/jpeg,width=1280,height=720 ! \
filesink location=frame.jpg
```

Capturing a Image to a File

```
gst-launch-1.0 \
v4l2src device=/dev/video0 num-buffers=1 ! \
video/x-raw,format=YUY2,width=640,height=480 ! \
videoconvert ! \
pngenc ! \
filesink location=frame.png
```

- pngenc - Encodes png images.
- Properties
  - Compression-level (Default-6)
  - snapshot

Capturing Multiple Images

```
gst-launch-1.0 \
v4l2src device=/dev/video0 num-buffers=10 ! \
image/jpeg,width=640,height=480 ! \
multifilesink location=frame-%02d.jpg
```

Recording a Video

```
gst-launch-1.0 \
v4l2src device=/dev/video0 num-buffers=300 ! \
image/jpeg,width=640,height=480,framerate=30/1 ! \
avimux ! filesink location=video.avi
```

Recording a Video

```
gst-launch-1.0 \
v4l2src device=/dev/video0 num-buffers=300 ! \
video/x-raw,format=YUY2,width=640,height=480,framerate=30/1 ! \
videoconvert ! \
x264enc tune=zerolatency ! \
mp4mux ! \
filesink location=video.mp4
```

- X264enc encodes raw video into H264 compressed data
- Properties
  - Bitrate (default 2048kbit/s)

Playing a Video

```
gst-launch-1.0 \
filesrc location=video.mp4 ! \
qtdemux ! \
avdec_h264 ! \
videoconvert ! \
autovideosink
```

Display Camera Stream

```
gst-launch-1.0 \
v4l2src device=/dev/video0 ! \
video/x-raw,format=YUY2,width=640,height=480,framerate=30/1 ! \
videoconvert ! \
autovideosink
```

Streaming via UDP (raw H64)

- Sender

```
gst-launch-1.0 \
v4l2src device=/dev/video0 ! \
video/x-raw,format=YUY2,width=640,height=480,framerate=30/1 ! \
videoconvert ! \
x264enc tune=zerolatency bitrate=2000 ! \
udpsink host=127.0.0.1 port=5000
```

- Reciever

```
gst-launch-1.0 \
udpsrc port=5000 ! \
h264parse ! \
avdec_h264 ! \
videoconvert ! \
autovideosink
```

RTP Packets via UDP

- Sender

```
gst-launch-1.0 \
v4l2src device=/dev/video0 ! \
video/x-raw,format=YUY2,width=640,height=480,framerate=30/1 ! \
videoconvert ! \
x264enc tune=zerolatency bitrate=2000 ! \
h264parse ! \
rtph264pay pt=96 ! \
udpsink host=127.0.0.1 port=5000
```

- Reciever

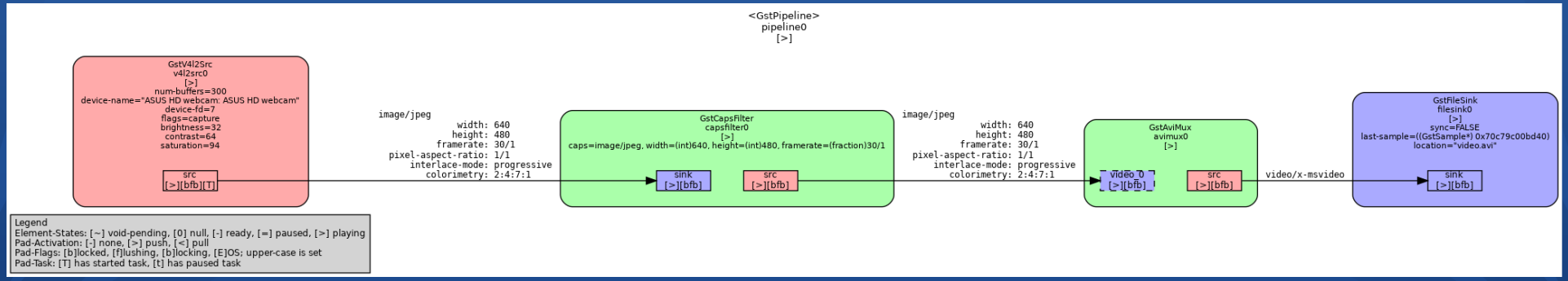
```
gst-launch-1.0 \
udpsrc port=5000 \
caps="application/x-rtp,media=video,encoding-name=H264,payload=96,clock-rate=90000" ! \
rtph264depay ! \
avdec_h264 ! \
videoconvert ! \
autovideosink
```

Streaming a File

```
gst-launch-1.0 \
filesrc location=video.mp4 ! \
qtdemux ! \
h264parse ! \
rtph264pay pt=96 ! \
udpsink host=127.0.0.1 port=5000
```

Dumping and Visualizing

- export GST_DEBUG_DUMP_DOT_DIR=/tmp/gstreamer_caps
- gst-launch-1.0 v4l2src device=/dev/video0 num-buffers=300 ! image/jpeg,width=640,height=480,framerate=30/1 ! avimux ! filesink location=video.avi
- dot -Tpng /tmp/gstreamer_caps/0.00.00.797349133-gst-launch.PAUSED_PLAYING.dot -o pipeline.png



Reference

- https://gstreamer.freedesktop.org/
- https://gstreamer.freedesktop.org/documentation/tools/index.html?gi-language=c